



AWS Glue

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

AWS Glue

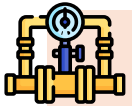


AWS Glue **serverless data integration service** that makes it easy for analytics users to discover, prepare, move, and integrate data from multiple sources.

Use cases for:

- Analytics
- machine learning
- application development

discover and connect to more than 70 diverse data sources and manage your data in a centralized data catalog.



Visually create, run, and monitor extract, transform, and load (ETL) pipelines to load data into your data lakes

Immediately search and query cataloged data using:

- Amazon Athena
- Amazon EMR
- Amazon Redshift Spectrum

What it can do:

- Data discovery
- Modern ETL or ELT
- Cleansing
- Transforming
- Centralized cataloging

AWS Glue Jobs

There are three engines for AWS Glue Jobs:

- Python Shell Engine
- Ray Job
- Spark Job

AWS Glue Jobs can be create in:

- Visual ELT (AWS Glue Studio)
- Jupyter Notebooks
- Script Editor (within AWS)

Create job [Info](#)



Author in a visual interface focused on data flow.

Visual ETL



Author using an interactive code notebook.

Notebook



Author code with a script editor.

Script editor

- AWS Glue ETL jobs are charged based on the number of data processing units (DPUs)
 - AWS Glue allocates 10 DPUs to each Spark job
 - 2 DPUs to each Spark Streaming job.
 - AWS Glue allocates 6 M-DPUs to each Ray job
- A **combination of Work Type and Number of Workers** will determine DPUs

Worker type

Set the type of predefined worker that is allowed when a job runs.

G 1X
(4vCPU and 16GB RAM)

Requested number of workers

The number of workers you want AWS Glue to allocate to this job.

10

AWS Glue Studio

AWS Glue Studio allows to visually build ETL pipelines.
Also known as the Visual ETL

A pipeline is composed of **nodes**:

Sources:

- The data you plan to use

Transforms:

- What you want to do to the data

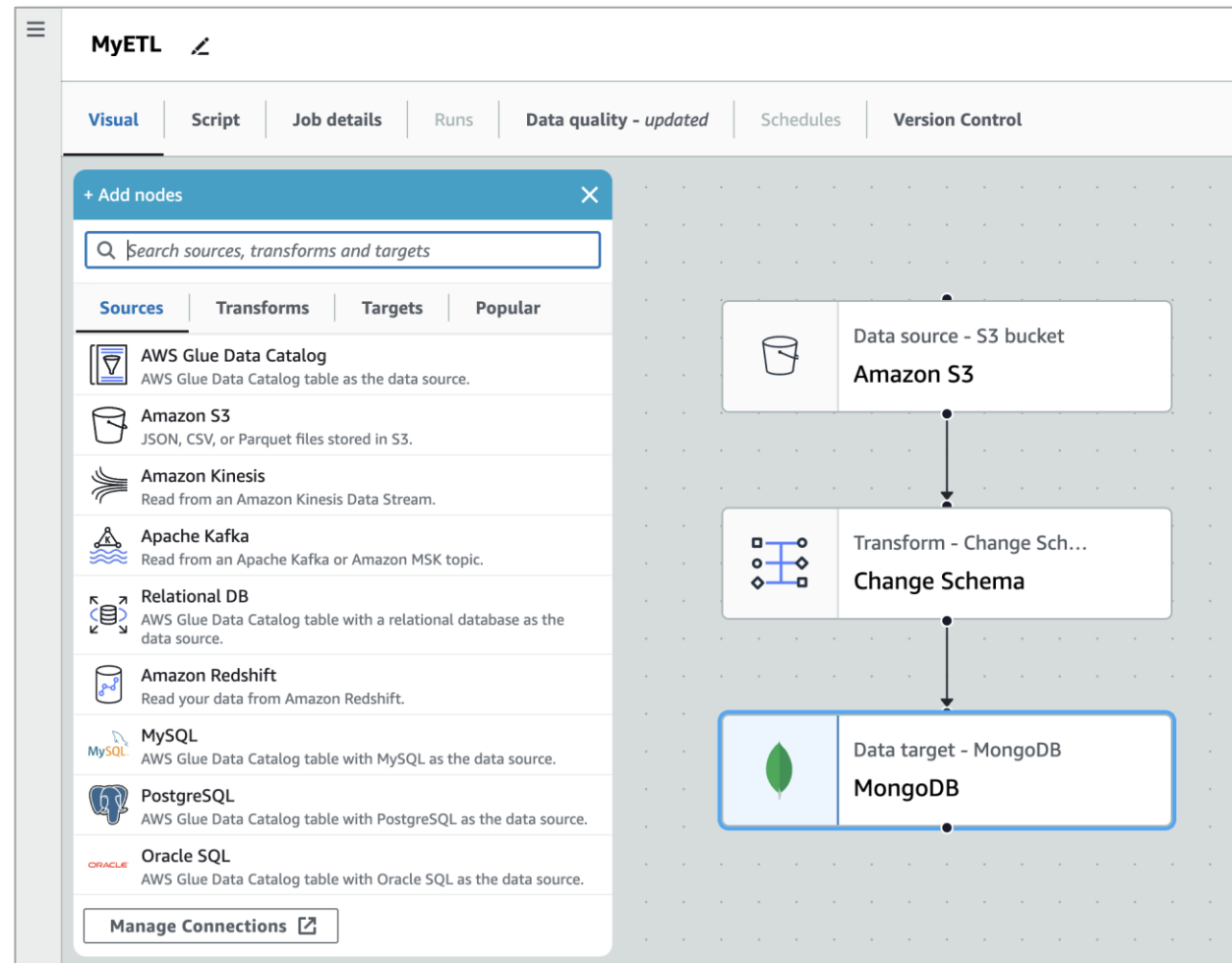
Targets:

- Where you want to send the data

You can **version control** your pipelines using:

- AWS Code Commit
- GitHub
- GitLab
- BitBucket

AWS Glue Studio is for visually preparing a Glue Job with little to no coding

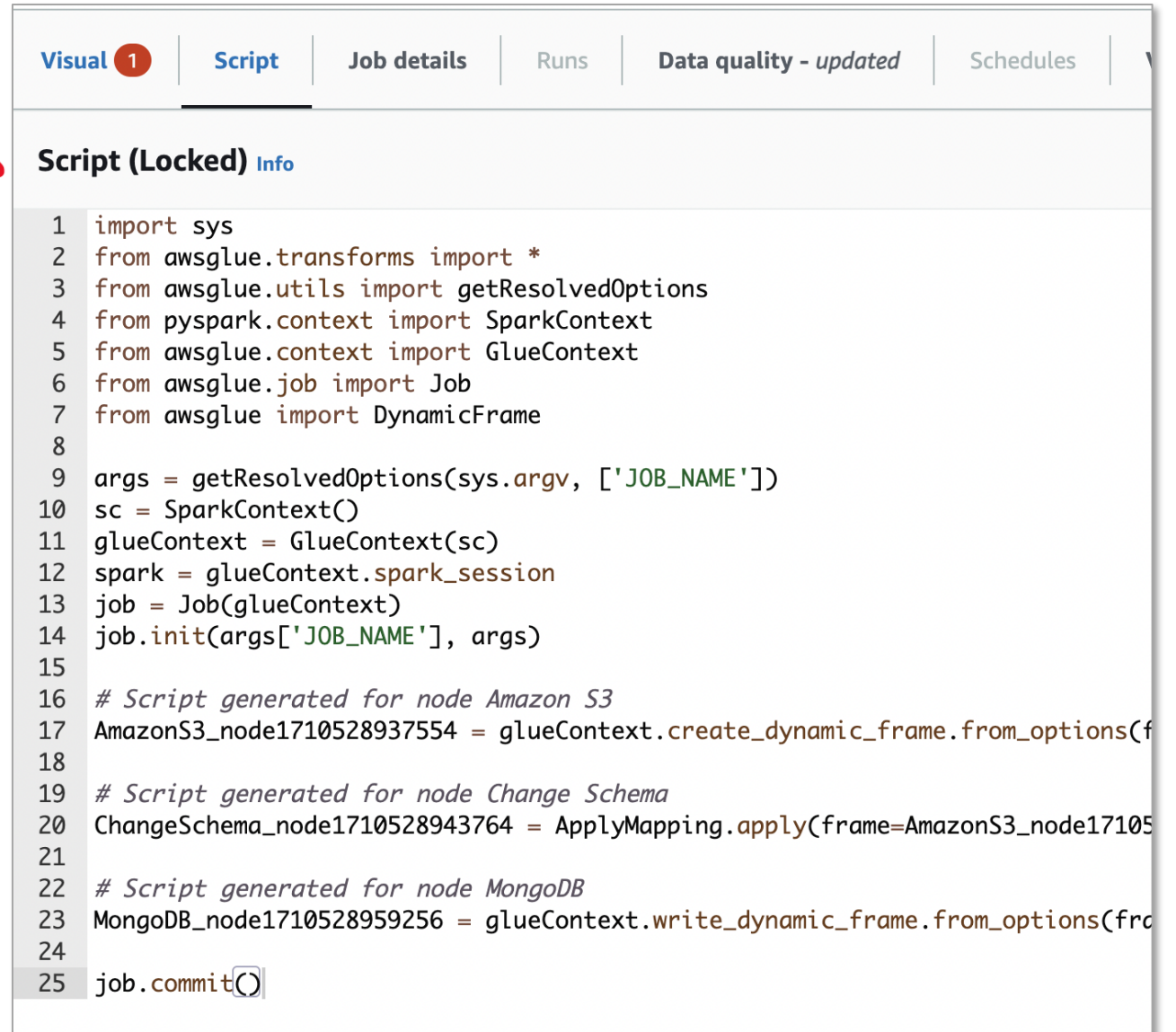


AWS Glue Studio

The Visual ETL will produce a python script that you can **download and execute using python** to run the script in your ELT tool or it will be used a job via AWS Glue jobs

You'll have to install the required AWS Glue Libraries

<https://github.com/aws-labs/aws-glue-libs>



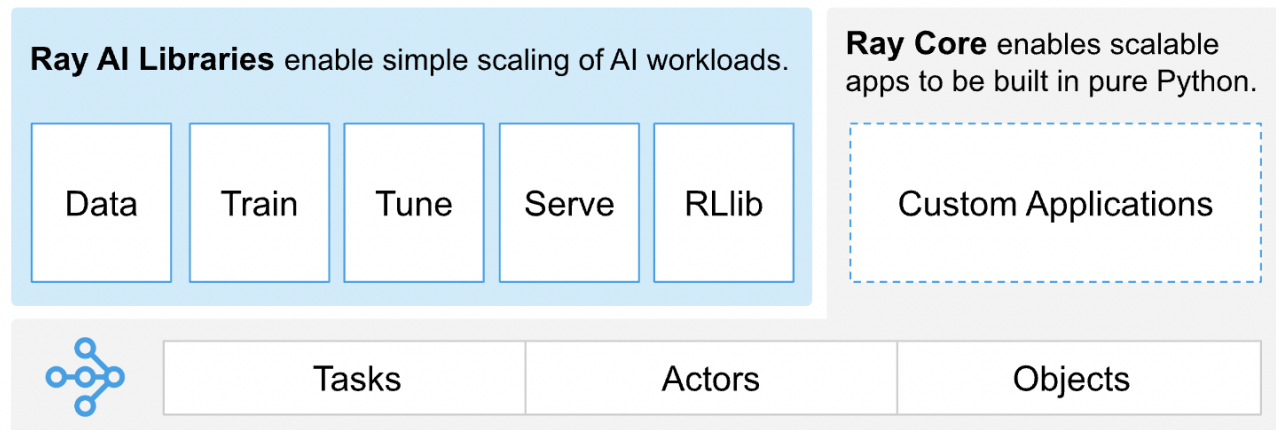
```
1 import sys
2 from awsglue.transforms import *
3 from awsglue.utils import getResolvedOptions
4 from pyspark.context import SparkContext
5 from awsglue.context import GlueContext
6 from awsglue.job import Job
7 from awsglue import DynamicFrame
8
9 args = getResolvedOptions(sys.argv, ['JOB_NAME'])
10 sc = SparkContext()
11 glueContext = GlueContext(sc)
12 spark = glueContext.spark_session
13 job = Job(glueContext)
14 job.init(args['JOB_NAME'], args)
15
16 # Script generated for node Amazon S3
17 AmazonS3_node1710528937554 = glueContext.create_dynamic_frame.from_options(fr
18
19 # Script generated for node Change Schema
20 ChangeSchema_node1710528943764 = ApplyMapping.apply(frame=AmazonS3_node17105
21
22 # Script generated for node MongoDB
23 MongoDB_node1710528959256 = glueContext.write_dynamic_frame.from_options(fro
24
25 job.commit()
```

Ray



Ray is a unified framework for scaling AI and Python applications and consists of a core distributed runtime and a set of AI libraries for simplifying ML compute.

<https://github.com/ray-project/ray>



- Ray has the advantage of being great for low-latency workloads where Apache Spark has not.
- Ray uses a decentralized metadata management system where Apache Spark is centralized.
- Ray has more fine-grade permissions compared to Apache Spark.
- Ray is a python-first library where Spark is built on top of the JVM

Apache Iceberg Table



Apache Iceberg is a open table format that has an **expressive SQL** designed to improve upon the limitations of traditional file formats like CSV, JSON, or Parquet files used in data lakes.

```
CREATE TABLE demo.nyc.taxis
(
  vendor_id bigint,
  trip_id bigint,
  trip_distance float,
  fare_amount double,
  store_and_fwd_flag string
)
PARTITIONED BY (vendor_id);

INSERT INTO demo.nyc.taxis
VALUES (1, 1000371, 1.8, 15.32, 'N'), (2, 1000372,
2.5, 22.15, 'N'), (2, 1000373, 0.9, 9.01, 'N'), (1,
1000374, 8.4, 42.13, 'Y');
```

- **Schema Evolution:** supports adding, deleting, updating, and renaming columns in data schema without impacting existing data.
- **Hidden Partitioning:** abstracts the complexity of partitioning away from users.
- **Snapshot Isolation:** tables are versioned, allowing full snapshot isolation between concurrent operations.
- **Incremental Processing:** Supports efficient incremental data processing.
- **Rollbacks and Time Travel:** allows users to roll back to previous table states or query historical snapshots of data for auditing or debugging purposes.
- **First-Class SQL Support:** Designed to work seamlessly with SQL-based analytics engines eg. Spark, Trino, Fink, Presto, Hive and Impala.
- **File Format Agnosticism:** can manage data stored in various file formats, including Parquet, ORC, and Avro
- **Efficient Upserts and Deletes:** Supports upserts (update or insert operations) and deletes without needing to rewrite entire data files.
- **Scalability:** Designed to handle petabyte-scale datasets with ease, making it suitable for large enterprises and data-intensive applications.
- **Speculative Execution Protection:** Prevents duplicate data due to speculative execution in distributed computing environments by uniquely identifying file writes.

Delta Lake



Delta Lake is an **open-source storage framework for building agnostic Lakehouse architecture**

Delta Lake is the **optimized storage layer that extends Parquet data files** with a file-based transaction log for ACID transactions, scalable metadata handling and Streaming and Batch Unification

Delta Lake has integration for

- Apache Spark
- PrestoDB
- Flink
- Trino
- Hive
- Snowflake
- Google BigQuery
- Athena
- Amazon Redshift
- Databricks
- Azure Fabric
- APIs for Scala, Java, Rust, and Python

Delta Lake has its own data structure called **Delta Tables** and stores data as **Deltas**.

AWS Glue Data Catalog



AWS Glue Data Catalog is a **fully managed, Apache Hive Metastore-compatible catalog service** that makes it easy for customers to store, annotate, and share metadata about their data.

Data Catalog is serverless, so its pay what you use.

AWS Glue Data Catalog integrates with:

- Amazon S3
- Amazon RDS
- Amazon Redshift
- Amazon Athena
- AWS Glue ETL
- Amazon EMR

There are two table formats:

- **Standard AWS Glue table**
 - You must specify the data format:
 - Avro, CSV, Json, XML, Parquet, ORC
 - Data can be sourced from:
 - Amazon S3, Kinesis, Kafka
- **Apache Iceberg table**
 - Uses its own expressive SQL data format

AWS Glue database is a container for multiple AWS Glue tables

AWS Glue table is the metadata definition that represents your data, including its schema. A table can be used as a source or target in a job definition.

AWS Glue Crawler can discover schema formats to define your AWS Glue tables.

AWS Glue Data Catalog – Crawlers



AWS Glue Data Crawler is a tool that is used to analyze a targeted data source to determine its schema and generate AWS Glue Data Tables.

Data Sources that Data Crawler can be connected to:

- Amazon S3
- Java Database Connectivity (JDBC)
 - Amazon RedShift
 - Snowflake
 - Amazon RDS
- DynamoDB
- MongoDB Client
 - MongoDB server, MongoDB Atlas, DocumentDB
- Delta Lake
- Apache Iceberg Tables stored in S3
- Hudi Tables stored in S3

Data Crawler can run on a schedule

Data Crawler can be run on demand

AWS Glue Data Quality



AWS Glue Data Quality allows you to measure and monitor the quality of your data so that you can make good business decisions.

Built on top of the AWS open-source DeeQu unit testing framework (which itself is built on top of Apache Spark unit tests)

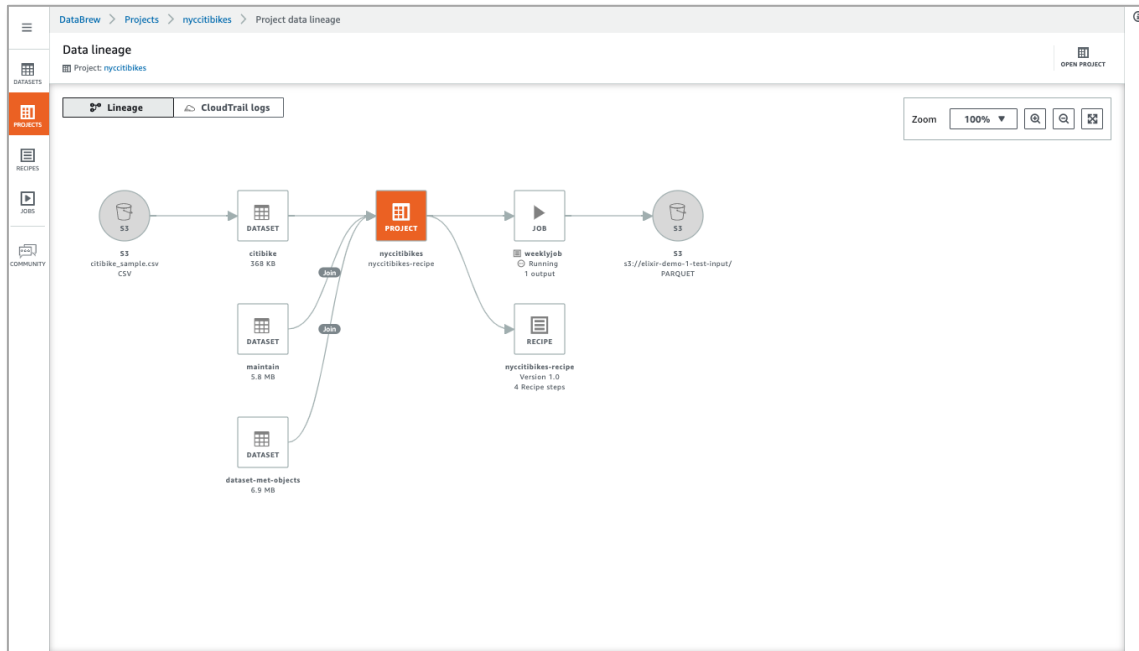
Uses **Data Quality Definition Language (DQDL)** - a domain-specific language to define data quality rules

- Use **machine learning (ML)** to detect anomalies and hard-to-detect data quality issues.
- Supports 25+ prebuilt **data quality (DQ) rules** and lets you create custom ones.
- Calculates a **Data Quality score** to help assess the health of your dataset.
- Identifies specific records that causes lower quality scores.
- Serverless – **you pay only for what you use**
- Can be used to enforce data quality checks on **AWS Glue ETL pipelines** and the **AWS Glue Data Catalog**

AWS Glue DataBrew



AWS Glue DataBrew is a visual data preparation tool that enables users to clean and normalize data without writing any code.



- helps reduce the time it takes to prepare data for analytics and machine learning (ML) by up to 80 percent
- choose from over 250 ready-made transformations to automate data preparation tasks
- can more easily collaborate to get insights from raw data
- Serverless offering so pay for what you use

Data Classification Scheme

Data Classification is when you organization data into classes or categories

Data Classification Scheme is **when these classes follow a specific pattern.**

For the security of your data you will want to define a Data Classification Scheme.

Washington, D.C Data Policy (2017) Scheme

- Level 0 — Open Data.
- Level 1 — Public Data, Not Proactively Released.
- Level 2 — For District Government Use.
- Level 3 — Confidential.
- Level 4 — Restricted Confidential.

US National Classification Scheme

- Confidential
- Secret
- Top Secret



AWS recommends starting with a **three-tiered data classification** approach

AWS Well Architected Framework of AWS Sustainability Pillar > Data Management

- **SUS04-BP01** Implement a data classification policy

You can use AWS Glue to Catalog and use **custom classifiers** to classify your data.